01 PRACTICAL 1

ASSIGNMENT 5 • CODE REVISION SHEET

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Lambda
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
```

```
# LOADING THE DATASET

DATA_DIR = "caltech-101"
IMG_SIZE = (224, 224)

# Load training data (80%)
train_ds = keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=32
)

# Load testing data (20%)
test_ds = keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=32
)

class_names = train_ds.class_names

print("Number of classes:", len(class_names))
```

```
# CHOOSING PRE-TRAINED MODEL - VGG16
base = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

# FREEING THE BASE LAYERS
base.trainable = False

# ADDING CUSTOM LAYERS
model = Sequential([
    Lambda(preprocess_input),
    base,
    Flatten(),
    Dense(256, activation="relu"),
    Dropout(0.5),   # Dropout helps prevent overfitting. -- randomly turns off 50% of
the neurons in that layer
    Dense(len(class_names), activation='softmax')
])

# COMPILE AND TRAIN THE MODEL
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_ds.take(50),
    validation_data=test_ds.take(50),
    epochs=3
)

# COMPUTE LOSS AND EVALUATE MODEL

loss, accuracy = model.evaluate(test_ds)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

# PLOT LOSS

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.legend()
plt.show()

# VISUALIZE PREDICTIONS

for images, labels in test_ds.take(1):

    predictions = model.predict(images)

    for i in range(5):
        predicted = np.argmax(predictions[i])
        actual = int(labels[i].numpy())

        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title("Actual: " + class_names[actual] +
                  "\nPredicted: " + class_names[predicted])
        plt.axis('off')
        plt.show()

# FINE TUNE THE MODEL

# Unfreeze VGG16
base.trainable = True

# Freeze all layers except last 4
for layer in base.layers[:-4]:
    layer.trainable = False

# Compile with small learning rate
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.00001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train again
model.fit(
    train_ds.take(50),
    validation_data=test_ds.take(50),
    epochs=3,
    batch_size = 32
```

```
)

# EVALUATION

loss, accuracy = model.evaluate(test_ds)

print("Final Loss:", loss)
print("Final Accuracy:", accuracy)
```

02 PRACTICAL 2

ASSIGNMENT 5 • CODE REVISION SHEET

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Lambda, Flatten
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
```

```
# LOADING DATASET
DATA_DIR = "Intel Image Classification"
IMG_SIZE = (224, 224)

# Loading training data (80%)
train_ds = keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split = 0.2,
    batch_size = 32,
    subset = "training",
    image_size = IMG_SIZE,
    seed = 42
)

# Loading testing data (20%)
test_ds = keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split = 0.2,
    batch_size = 32,
    subset = "validation",
    image_size = IMG_SIZE,
    seed = 42
)

class_names = train_ds.class_names

print("Number of classes: ", len(class_names))
print(class_names)
```

```
# CHOOSING THE PRE-TRAINED MODEL - VGG16
base_model = VGG16(
    weights = "imagenet",
    include_top = False,
    input_shape = (224,224,3)
)

# FREEZING THE BASE LAYERS AND ADDING CUSTOM LAYERS
base_model.trainable = False

# ADDING CUSTOM LAYERS
model = Sequential([
    Lambda(preprocess_input),
    base_model,
    Flatten(),
    Dense(256, activation="relu"),
    Dropout(0.5),
    Dense(len(class_names), activation="softmax")
])

model.summary()
```

```
# COMPILE AND TRAIN THE MODEL
model.compile(
    optimizer = 'adam',
    loss = "sparse_categorical_crossentropy",
    metrics = ['accuracy']
)

history = model.fit(
    train_ds.take(10),
    validation_data = test_ds.take(10),
    epochs = 3
)

# COMPUTE LOSS AND EVALUATE MODEL
test_loss , test_accuracy = model.evaluate(test_ds.take(10))

print("Test Loss: ", test_loss)
print("Test Accuracy: " , test_accuracy)

# PLOT LOSS
plt.plot(history.history['loss'], label = "Training loss")
plt.plot(history.history['val_loss'], label = "Validation Loss")
plt.legend()
plt.show()

# VISUALIZE PREDICTIONS

for images, labels in train_ds.take(1):
    predictions = model.predict(images)

    for i in range(10):
        predicted = np.argmax(predictions[i])
        actual = int(labels[i].numpy())

        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title("Actual: " + class_names[actual] +
                  "\nPredicted: " + class_names[predicted])
        plt.show()

# FINE TUNE THE MODEL

# Unfreeze VGG16
base_model.trainable = True

# Freeze all layers except last 4
for layer in base_model.layers[:-4]:
    layer.trainable = False

# Compile with small learning rate
model.compile(
    optimizer = 'adam',
    loss = 'sparse_categorical_crossentropy',
    metrics = ['accuracy']
)

model.fit(
    train_ds.take(10),
    validation_data=test_ds.take(10),
    epochs=3
)
```

03 PRACTICAL 3

ASSIGNMENT 5 • CODE REVISION SHEET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Lambda, Dropout
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input

# LOADING THE DATASET
DATA_DIR = "fashion mnist"
IMG_SIZE = (224, 224)

# loading training data (80%)
train_ds = keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split = 0.2,
    seed = 42,
    batch_size = 32,
    image_size = IMG_SIZE,
    subset = "training"
)

# loading testing data (20%)
test_ds = keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split = 0.2,
    seed = 42,
    subset = "validation",
    batch_size= 32,
    image_size = IMG_SIZE
)

class_names = train_ds.class_names

print(len(class_names))
print(class_names)

# CHOOSING PRE-TRAINED BASE MODEL - VGG16
base_model = VGG16(
    weights = "imagenet",
    include_top = False,
    input_shape=(224, 224, 3)
)

# FREEZING THE BASE LAYER
base_model.trainable = False

# ADDING CUSTOM LAYER
model = Sequential([
    Lambda(preprocess_input),
    base_model,
    Flatten(),
    Dense(256, activation="relu"),
    Dropout(0.5),
    Dense(len(class_names), activation="sigmoid")
])

# COMPILE AND TRAIN THE MODEL
model.compile(
```

```
    optimizer = 'adam',
    loss = "sparse_categorical_crossentropy",
    metrics = ['accuracy']
)

history = model.fit(
    train_ds.take(10),
    validation_data = test_ds.take(10),
    epochs = 3
)

# COMPUTE AND EVALUATE MODEL
test_loss, test_accuracy = model.evaluate(test_ds.take(10))

print("Test loss: ", test_loss)
print("Test Accuracy: ", test_accuracy)

# VISUALIZE THE PREDICTIONS

for images, labels in test_ds.take(1):
    predictions = model.predict(images)

    for i in range(32):
        predicted = np.argmax(predictions[i])
        actual = int(labels[i].numpy())

        if ( class_names[actual] != class_names[predicted]):
            plt.imshow(images[i].numpy().astype("uint8"))
            plt.title("Actual: " + class_names[actual] + "\nPredicted: " + class_names[
predicted])
            plt.show()

# FINE TUNE THE MODEL

# Unfreeze VGG16
base_model.trainable = True

# Freeze all layers except last 4
for layer in base_model.layers[:-4]:
    layer.trainable = False

# Compile model
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.00001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train again
model.fit(
    train_ds.take(10),
    validation_data=test_ds.take(10),
    epochs=3,
    batch_size = 32
)
```